UNIX Socket Programming

A Deep Dive into Socket API Functions

TCP/IP | UDP | Client-Server | Network I/O

Dr. Shailesh Sivan

Overview

What we'll cover today

1. `socket()`
2. `bind()`
3. `listen()`
4. `accept()`
5. `connect()`
6. `send() / recv()`
7. `sendto() / recvfrom()`
8. `close()`

UNIX Socket API

TCP (Stream)

Connection-oriented, reliable, ordered data transfer. Used in the Matrix Type Server.

UDP (Datagram)

Connectionless, fast, best-effort delivery. Used in the Sentence Translator.

Client-Server Flow

```
socket → bind → listen → accept  ←  server side
socket → connect → send/recv  ←  client side
```

socket()

CORE

Creating the communication endpoint

```
#include <sys/socket.h>
int sockfd = socket(int domain, int type, int protocol);
```

domain

`AF_INET, AF_INET6, AF_UNIX`

Specifies protocol family. AF_INET = IPv4, AF_INET6 = IPv6.

type

`SOCK_STREAM, SOCK_DGRAM`

SOCK_STREAM → TCP (reliable).
SOCK_DGRAM → UDP (fast).

protocol

`0 (auto-select)`

Usually 0; system selects correct protocol for domain+type.

Return value: Returns a file descriptor (int ≥ 0) on success, **-1** on error.
From the programs:
`sockfd = socket(AF_INET, SOCK_DGRAM, 0);` *(UDP Translator)*

💡 Always check return value and call close(sockfd) when done.

bind() & listen()

SERVER

Assigning address and enabling incoming connections

bind()

```
int bind(int sockfd,
  const struct sockaddr *addr,
  socklen_t addrlen);
```

Purpose: Associates the socket with a specific IP address and port number.

Key fields of sockaddr_in:
- **sin_family:** AF_INET
- **sin_addr.s_addr:** INADDR_ANY
- **sin_port:** htons(PORT)

```
From server.c (TCP):
bind(sockfd, (struct sockaddr *)&addr,
sizeof(addr))
```

listen()

```
int listen(int sockfd, int backlog);
```

Purpose: Marks the socket as passive — it will accept incoming connections (TCP only).

sockfd: Socket returned by socket()

backlog: Max pending connections queue (BACKLOG=5 in the server.c)

UDP note: listen() is NOT used for UDP sockets — connectionless!

```
From server.c (TCP):
listen(server_fd, BACKLOG)  /* BACKLOG = 5 */
```

accept() & connect()

Establishing the connection between server and client

accept() — Server Side

```
int accept(int sockfd,
  struct sockaddr *addr,
  socklen_t *addrlen);
```

Purpose: Extracts the first pending connection from the queue, creates a NEW connected socket, and returns its descriptor.

- Blocks until a client connects
- Returns new fd for that specific client
- addr filled with client's IP:port info
- Original socket continues to listen

```
client_fd = accept(server_fd,
  (struct sockaddr *)&addr, &addr_len);
```

connect() — Client Side

```
int connect(int sockfd,
  const struct sockaddr *addr, socklen_t len);
```

Purpose: Initiates a connection to the server at the specified address and port.

- For TCP: performs 3-way handshake
- For UDP: sets default destination only
- Blocks until connected or error
- Returns 0 on success, -1 on error

```
connect(sockfd,
  (struct sockaddr *)&server_addr,
sizeof(server_addr));
```

send() & recv()

TCP ONLY

Data transfer over TCP (connected sockets)

send()

```
ssize_t send(int sockfd,
  const void *buf, size_t len, int flags);
```

Sends data through a connected socket.

flags: Usually 0. Can use MSG_NOSIGNAL, MSG_DONTWAIT

Returns: Number of bytes actually sent. May be less than len!

send_all() Pattern (from client.c):

```
while (total < len) {
  sent = send(sock, ptr+total, len-total, 0);
  total += sent;
}
```

recv()

```
ssize_t recv(int sockfd,
  void *buf, size_t len, int flags);
```

Receives data from a connected socket.

Returns: **> 0** bytes received | **0** = connection closed | **-1** = error

```
Used in TCP server.c for matrix data:
recv_all(client_fd, &value_net, sizeof(value_net))
```

⚠ TCP is a stream! Always loop recv() to guarantee all expected bytes are received — use recv_all() pattern.

sendto() & recvfrom()

UDP

Connectionless data transfer — used by UDP sockets

sendto()

```
ssize_t sendto(int sockfd,
  const void *buf, size_t len, int flags,
  const struct sockaddr *dest, socklen_t dlen);
```

Sends a datagram to a specific destination. Unlike send(), you specify the recipient's address each time.

Extra params vs send():

dest_addr — target IP:port
addrlen — size of dest_addr

```
From UDP client.c:
sendto(sockfd, buffer, len, 0,
  (struct sockaddr*)&server_addr,
sizeof(server_addr));
```

recvfrom()

```
ssize_t recvfrom(int sockfd,
  void *buf, size_t len, int flags,
  struct sockaddr *src, socklen_t *srclen);
```

Receives a datagram and also captures the sender's address. Essential for UDP servers to know who to reply to.

src_addr → filled with sender's IP:port
addrlen → in/out: size of src_addr

```
From UDP server.c:
recvfrom(sockfd, buffer, BUFFER_SIZE-1, 0,
  (struct sockaddr*)&client_addr, &client_len);
```

close() & Helper Functions

Cleanup and address utilities

close()

```
int close(int fd);   /* #include <unistd.h> */
```

Closes the socket fd, releasing system resources.

- Always call when done — prevents resource leaks
- For TCP: triggers FIN/ACK graceful shutdown
- Both programs close(sockfd) at exit

inet_pton()

```
int inet_pton(int af, const char *src,
  void *dst);
```

Converts a human-readable IP string ("127.0.0.1") into binary form for sockaddr_in.sin_addr. Used in client.c.

htons() / ntohs()

```
uint16_t htons(uint16_t host);
```

Converts port numbers between host and network byte order. Always use htons(PORT) before storing in sin_port.

inet_ntoa() / memset()

```
char *inet_ntoa(struct in_addr);
```

inet_ntoa() converts binary IP back to string (for logging). memset() zeroes sockaddr structs before use — always do this!

TCP vs UDP: Socket Function Comparison

As demonstrated by the Matrix Server (TCP) and Sentence Translator (UDP) programs

Function	TCP (SOCK_STREAM)	UDP (SOCK_DGRAM)	Used In
`socket()`	AF_INET, SOCK_STREAM	AF_INET, SOCK_DGRAM	Both
`bind()`	Required on server	Required on server	Both servers
`listen()`	✓ Required	✗ Not used	TCP server
`accept()`	✓ Required	✗ Not used	TCP server
`connect()`	3-way handshake	Sets default addr	TCP client
`send/recv()`	✓ After connect	✗ Not typical	TCP client/server
`sendto/recvfrom()`	Can be used	✓ Primary method	UDP client/server
`close()`	Triggers FIN/ACK	Releases fd only	Both

Note: setsockopt(SO_REUSEADDR) is used in the TCP server to allow quick restart without "address already in use" errors.

Summary

`socket()`
Creates the endpoint; sets protocol family and type

`bind()`
Assigns IP address + port to the socket

`listen()`
Switches socket to passive mode (TCP only)

`accept()`
Accepts new client; returns a per-client fd

`send/recv()`
Reliable data exchange over TCP streams

`sendto/recvfrom()`
Datagram exchange with address info for UDP

UNIX Socket Programming • socket() → bind() → listen() → accept() → send/recv() → close()

Constants & Macros

`#define`

Predefined values used in UNIX socket programming

Address Family (domain)

`AF_INET`	2	IPv4 Internet protocols. Used in both programs.
`AF_INET6`	10	IPv6 Internet protocols.
`AF_UNIX`	1	Local (inter-process) communication.
`INADDR_ANY`	0.0.0.0	Bind to all available interfaces (server).

Socket Type (type)

`SOCK_STREAM`	TCP — reliable, connection-oriented, ordered stream. Matrix server uses this.
`SOCK_DGRAM`	UDP — connectionless, fast datagrams. Sentence translator uses this.
`SOCK_RAW`	Raw socket access — low-level packet control.

Program-Defined Constants (from your programs)

`SERVER_IP`	"127.0.0.1"	Loopback address — server runs on same machine as client.	`PORT`	7777 / 7778	Port number the server listens on. 7777 = TCP matrix, 7778 = UDP translator.
`BUFFER_SIZE`	1024 / 2048	Size of I/O buffers for send/recv. Prevents overflow on stack-allocated arrays.	`BACKLOG`	5	Maximum pending TCP connections queued before server calls accept().

Common flags: 0 (default) | MSG_NOSIGNAL | MSG_DONTWAIT | MSG_WAITALL — passed as 4th arg to send()/recv()/sendto()/recvfrom()

Key Structures

`struct`

Data structures for addressing and socket configuration

`struct sockaddr_in`

```
struct sockaddr_in {
  sa_family_t  sin_family;  /* AF_INET */
  in_port_t    sin_port;    /* htons(PORT) */
  struct in_addr sin_addr;  /* INADDR_ANY */
  char         sin_zero[8]; /* padding */
};
```

`sin_family`	`sa_family_t`	Address family — always AF_INET for IPv4.
`sin_port`	`in_port_t`	Port in network byte order. Use htons(PORT) to convert.
`sin_addr`	`struct in_addr`	IPv4 address. Set to INADDR_ANY (server) or inet_pton() result (client).
`sin_zero`	`char[8]`	Padding — zero it with memset() before use.

`struct sockaddr`

```
struct sockaddr {
  sa_family_t sa_family;
  char        sa_data[14];
};
```

Generic address structure. All socket functions accept (struct sockaddr *). Cast sockaddr_in to this type when passing to bind(), connect(), accept().

`socklen_t`

typedef uint32_t socklen_t;
Used to specify the size of address structures. Passed by pointer to accept() and recvfrom() so the kernel can write back the actual address size.

`struct in_addr`

struct in_addr { uint32_t s_addr; }; Holds a 32-bit IPv4 address. Populated by inet_pton() on the client side.